Exercício 1

Crie uma classe Lampada com os seguintes atributos de instância:

- divisao: nome da divisão onde se encontra.
- ligada: indica se está ligada. O estado inicial deve ser False.

Implemente os métodos:

Método	Comportamento
ligar()	Coloca a lâmpada no estado ligado
desligar()	Coloca a lâmpada no estado desligado
descricao()	Devolve uma string com a divisão e o estado atual

O método descricao() deve produzir mensagens como "Sala: ligada" ou "Quarto: desligada".

```
sala = Lampada("Sala")
quarto = Lampada("Quarto")

sala.ligar()

assert sala.ligada is True
assert quarto.ligada is False
assert sala.descricao() == "Sala: ligada"

sala.desligar()
assert sala.descricao() == "Sala: desligada"
```

Exercício 2

Crie uma classe CarteiraDigital com:

- titular: nome do titular.
- saldo: valor disponível, inicialmente zero, salvo indicação em contrário.

Implemente:

Método	Comportamento
depositar(valor)	Acrescenta um valor positivo ao saldo
pagar(valor)	Retira um valor positivo, desde que exista saldo suficiente
consultar_saldo()	Devolve o saldo sem o alterar

Regras:

1. O saldo inicial não pode ser negativo.
2. Depósitos e pagamentos devem ter valores positivos.
3. Um pagamento não pode ultrapassar o saldo disponível.
4. Uma operação inválida deve lançar ValueError sem alterar o saldo.

```
carteira = CarteiraDigital("Ana", 100)

carteira.depositar(50)
carteira.pagar(30)

assert carteira.consultar_saldo() == 120
assert carteira.consultar_saldo() == 120

try:
    carteira.pagar(200)
except ValueError:
    print("Pagamento recusado.")
else:
    raise AssertionError("O pagamento deveria ter sido recusado.")

assert carteira.consultar_saldo() == 120
```

Exercício 3

Crie uma classe Turma com:

- Um atributo de classe escola, inicialmente "Escola ABC".
- Um atributo de instância nome.
- Uma lista de instância alunos, inicialmente vazia.

Implemente:

Método	Comportamento
adicionar_aluno(nome)	Acrescenta um nome à lista da turma
numero_alunos()	Devolve o número de alunos dessa turma

Cada turma deve ter a sua própria lista. A escola, pelo contrário, deve ser inicialmente consultada através do atributo da classe.

```
turma_a = Turma("A")
turma_b = Turma("B")

turma_a.adicionar_aluno("Ana")
turma_a.adicionar_aluno("Bruno")
turma_b.adicionar_aluno("Carla")

assert turma_a.alunos == ["Ana", "Bruno"]
assert turma_b.alunos == ["Carla"]
assert turma_a.alunos is not turma_b.alunos

assert turma_a.numero_alunos() == 2
assert turma_b.numero_alunos() == 1

Turma.escola = "Escola XYZ"

assert turma_a.escola == "Escola XYZ"
assert turma_b.escola == "Escola XYZ"
```

O que poderia acontecer se alunos = [] fosse definido apenas no corpo da classe?